Hamro Hospital Management System (Nepal) - Checkpoint 2 (Full System)

A Django 5 Hospital Management System built for hospitals in Nepal - now with **all 14 modules integrated into one project**.

What's included

Fully working (models, forms, views, URLs, templates, admin - no placeholders):

- **Public website** - Home, About, Departments, Doctors, Diseases, Gallery, Contact
- **Accounts & Roles** - custom User model with 8 roles, role-based access decorators, append-only system-wide Audit Trail, staff login/logout, Super Admin staff management
- **Departments** - full CRUD (Super Admin)
- **Doctors** - full CRUD, schedule, photo, consultation fee (Super Admin)
- **Patients** - Nepal address system (all 7 provinces / 77 districts, seeded and searchable), auto-generated Patient ID (`HT-2026-000001`), QR code generation, OPD Visit with auto token numbers and auto registration fee, printable OPD Ticket and Patient Card, 24-hour free-edit window with audit logging
- **Appointments** - public online booking with a complete eSewa ePay v2 integration (HMAC-SHA256 signed requests, callback verification), printable receipt with QR
- **Consultations** - Doctor's queue, diagnosis, dynamic prescription builder, lab/radiology test requests, full patient consultation history
- **Laboratory** - request queue, result upload, printable lab report
- **Radiology** - request queue (X-Ray/CT/MRI/ECG/Echo/Ultrasound/Custom), findings & impression, printable report, its own `RAD-2026-000001` numbering
- **Admissions** - Ward & Bed management, admit/discharge workflow with automatic bed-freeing on discharge, length-of-stay tracking
- **Billing (Cash Counter)** - patient lookup, multi-service billing with automatic

totals, Cash/eSewa/Insurance payment, audit-safe price snapshotting, printable receipt, separately-audited reprint, today's collections
- **Pharmacy + Inventory** - medicine catalogue with low-stock/out-of-stock/near-expiry/expired flags, atomic stock adjustments, dispensing against digital/manual/walk-in prescriptions with live stock validation, printable pharmacy bill
- **Insurance** - insurer directory (Super Admin), claim submission and approve/reject/settle review workflow, claims report
- **Reports** - Revenue Dashboard (daily/monthly/yearly), Registration, Department, Doctor, Cash Counter, Pharmacy, and Laboratory reports
- **Patient Portal** - self-service signup/login on the main public website using Hospital ID + phone number + password; logged-in patients can view all their visits, prescriptions, lab reports, radiology reports, admissions, bills, pharmacy purchases, and insurance claims, and can **book a new visit or follow-up themselves** - this creates the same Visit/receipt/token/OPD ticket a Registration Counter staff member would create, reusing the identical ticket template and QR code

Requirements

- Python 3.12+ (developed against 3.12; should also work on 3.10/3.11)
- pip

Setup (Windows / Linux / macOS)

```bash
python -m venv venv

# Windows
```

```
venv\Scripts\activate

# Linux / macOS
source venv/bin/activate

pip install -r requirements.txt

python manage.py migrate

# ONE command builds the entire interconnected demo hospital:
# 500 patients, 150 doctors, 5000 appointments, 5000 visits/consultations/
# prescriptions, ~4000 bills, 4000 pharmacy sales, 500 admissions,
# 300 insurance claims, medicines, lab/radiology catalogues, demo staff
# accounts for every role, etc. Takes 5-10 minutes; safe to re-run.
python manage.py seed_all

python manage.py runserver
```

Then open **http://127.0.0.1:8000/** for the public site, and **http://127.0.0.1:8000/accounts/login/** to sign in as staff using any of the demo accounts below (all created automatically by `seed_all`) - no `createsuperuser` step needed, though you're welcome to run `python manage.py createsuperuser` too if you'd rather use your own login.

Creating other staff roles

Log in as Super Admin -> **Staff Accounts** -> **Add Staff Member**, and pick a role. All 8 roles now have a working dashboard: Registration Counter, Cash Counter, Doctor, Pharmacy, Laboratory, Radiology Counter, Insurance Counter, and Super Admin.

Setting up billable services & medicines

Hospital Services (used by Billing) now have a full Super Admin screen at **Dashboard -> Hospital Services** (add/edit code, name, department, price). **Medicines** (used by Pharmacy) have their own screen at **Dashboard -> Medicines**. Both are also visible in Django Admin at **/admin/** if you prefer bulk editing there.

Patient Portal signup

Patients can create their own portal login at **/portal/signup/** (also linked from the "Patient Login" navbar item on the public site). Signup requires the patient's Hospital ID (e.g. `HT-2026-000001`) and the phone number already on their patient record - so a patient must already be registered by the Registration Counter at least once before they can sign up. This is a separate login system from staff accounts; it does not use Django's built-in auth system.

Project layout

```
tu_hospital_management_system/
 ├── manage.py
 ├── requirements.txt
```

```
├── README.md
├── .gitignore
├── .env.example
├── config/          settings, urls, wsgi/asgi
├── accounts/        custom User, roles, audit log, staff management
├── website/         public site + Hospital Services catalogue
├── departments/     department CRUD
├── doctors/         doctor CRUD
├── patients/        Nepal address system, patient registration, OPD visits, QR
├── appointments/    online booking + eSewa integration
├── consultations/   doctor workflow: diagnosis, prescriptions, lab/radiology requests
├── laboratory/      lab request queue and results
├── radiology/       imaging request queue and reports
├── admissions/      ward/bed management, admit/discharge
├── billing/         Cash Counter billing and receipts
├── pharmacy/        medicine catalogue, inventory, dispensing
├── insurance/       insurer directory, claims workflow
├── reports/         revenue dashboard and per-module reports
├── patient_portal/  patient self-service signup/login, records, self-booking
├── templates/
├── static/
└── media/           uploaded photos, QR codes (created at runtime)
```

Demo Accounts

Created automatically by `seed_all` (or run `python manage.py create_demo_accounts` on its own to (re)create/reset them):

Same password for **every** account: **`password123*#`**

Username	Role
admin	Super Admin
registration	Registration Counter
cashier	Cash Counter
doctor	Doctor
pharmacy	Pharmacy
laboratory	Laboratory
radiology	Radiology
insurance	Insurance
admission	Ward/Admission
nursing	Nursing
operationtheatre	Operation Theatre
bloodbank	Blood Bank
accounts	Accounts Dept
medicalrecords	Medical Records

Safe to re-run — updates password/role instead of duplicating if accounts already exist.

Notes

- Each module (Billing, Pharmacy, Laboratory, Admission, Insurance) now generates
  its own unique rectangular barcode on its own records, separate from the
  Patient's square QR code. Run `pip install -r requirements.txt` (adds

`python-barcode`) then `python manage.py migrate` to pick this up.

- Uses SQLite by default - zero extra database setup needed.
- `DEBUG=True` by default for local development; set `DJANGO_DEBUG=False` and a real `DJANGO_SECRET_KEY` before any real deployment.
- All sequential IDs (Patient, Visit receipt, Appointment, Bill, Pharmacy sale, Insurance claim, Admission, Radiology request) are generated inside atomic, row-locked transactions so concurrent activity across counters can never collide.
- eSewa credentials in `.env.example` are the public eSewa **test/sandbox** merchant code and secret - replace with your live merchant credentials before going live, and double-check eSewa's current merchant documentation, since payment gateway APIs can change.